Seal

A Technical Reference

The End-to-End Encrypted Chat Server

Rust • Axum • LanceDB • libsodium • Tauri

Architecture, Code Walkthrough, API Reference,
Security Model, Testing, and Operations

Generated July 2, 2026

Contents

Chapter 1

Introduction

1.1 What Seal Is

Seal is a fully end-to-end encrypted (E2EE) chat application. It offers direct messages (DMs) and group channels, an attachment path for encrypted images, and both a browser client and a native desktop client. The component documented in this reference is the *server*: a single, self-contained Rust binary named `seal-server` built on the Axum web framework, with persistence provided by LanceDB, an embedded columnar/vector database built on Apache Arrow.

The defining property of Seal is that **the server never sees plaintext**. All encryption and decryption happens on the client using libsodium. The server stores and relays opaque ciphertext, public keys (which are, by definition, public), and routing metadata (who sent what to whom, and when). Private keys never leave the client. This makes Seal a *zero-knowledge relay*: a database or server compromise exposes ciphertext and social graph metadata, but not message contents.

1.2 Design Goals

- **Zero-knowledge server.** The server is a dumb, trustworthy pipe. It authenticates users, enforces channel membership, rate-limits abuse, and persists ciphertext — nothing more. It holds no decryption keys.
- **Single static binary.** `templates/`, `static/`, and the default `config.yaml` are compiled *into* the binary via `include_dir!` / `include_str!`. The release artifact at `target/release/seal-server` can be copied anywhere, given a `JWT_SECRET` and a `DATABASE_PATH`, and run.
- **Faithful port.** The server is a Rust port of an earlier FastAPI (Python) application. Many functions carry comments such as "Mirrors `app/main.py`" precisely because behaviour — error strings, status codes, bcrypt hash format — was kept wire-compatible with the Python original and its test suite.
- **Testability.** The bootstrap path is factored so integration tests spin up the *real* router against a throwaway database on a random port. The suite exercises the HTTP and WebSocket surface end to end.

1.3 Technology Stack

Concern	Technology
HTTP / routing / WebSocket	Axum 0.8 on Tokio
Persistence	LanceDB 0.30 (Apache Arrow, `arrow-array` 58)
Password hashing	bcrypt (`DEFAULT_COST`)
Session tokens	JSON Web Tokens (HS256/384/512)
Client-side crypto	libsodium (X25519 + secretbox), in the browser/desktop
Config	YAML (`serde_yml`) with environment-variable overrides
Asset bundling	`include_dir` / `include_str`
Logging	`tracing` + `tracing-subscriber`
Desktop client	Tauri v2

1.4 A Note on Scope and Branch

This reference documents the code on the `main` branch. The repository also carries a `feat/object-storage` feature branch that teaches the database layer to talk to S3, GCS, and Azure Blob Storage instead of the local filesystem. That work is not yet merged into `main`; on `main` the database connector is filesystem-only. Chapter 11 describes the object-storage feature and clearly marks which parts live only on the feature branch, so the picture is complete without misrepresenting what `main` actually compiles.

1.5 How to Read This Book

- Chapter 2 gives the 30,000-foot view: the three-layer model, `AppState`, and a request's lifecycle.
- Chapters 3–6 drill into the subsystems: configuration, the database, the REST API, and WebSockets.
- Chapter 7 explains the cryptographic model and the server-side defences (auth, rate limiting, validation).
- Chapter 8 is a module-by-module code walkthrough — the reference you reach for when editing a specific file.
- Chapters 9–11 cover the test suite, build and deployment, and the object-storage backend.

Throughout, inline identifiers such as `db_ops::scan_where` and file paths such as `src/routes/messages.rs` point directly at the source so you can jump from prose to code.

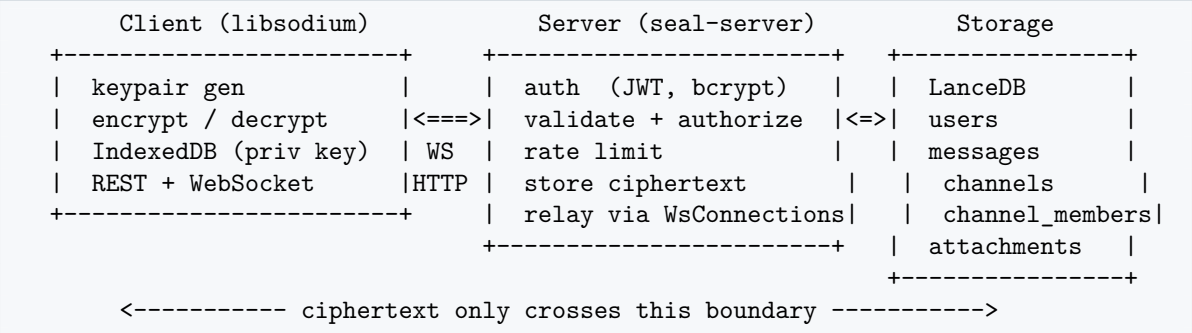
Chapter 2

Architecture

2.1 The Three-Layer Model

Seal is best understood as three layers with a strict trust boundary between the first two:

1. **Client layer** (browser or Tauri desktop). Generates the user's X25519 key pair, stores the private key locally (e.g. IndexedDB), and performs *all* encryption and decryption with libsodium. It speaks to the server over HTTP (REST) and a WebSocket.
2. **Server layer** (`seal-server`, this codebase). A stateless relay: it authenticates requests, validates and authorizes them, stores ciphertext, and fans messages out to connected recipients. It never has the keys to read message bodies.
3. **Storage layer** (LanceDB). Five Arrow-backed tables persisted to the local filesystem (on `main`) or to cloud object storage (on the `feat/object-storage` branch).



2.2 The Crate Layout

The project is a single Cargo crate that produces one binary. The split between `src/main.rs` and `src/lib.rs` exists so tests can link against the library and construct the same router the binary runs.

File	Responsibility
`src/main.rs`	Binary entry point: init tracing, load config, connect and initialize the DB, build `AppState`, bind the listener, serve.
`src/lib.rs`	Library root: declares modules, defines `AppState` and `build_router`, serves the embedded `index.html` and static assets.

File	Responsibility
`src/config.rs`	Layered configuration (embedded YAML → disk YAML → env vars); compiles validation regexes.
`src/db.rs`	Arrow schemas for the five tables; `connect` and `init_db`; the messages-table migration.
`src/db_ops.rs`	Thin helpers over LanceDB tables: scans, appends, row builders, and column extractors.
`src/models.rs`	Serde request/response structs.
`src/auth.rs`	bcrypt hashing and JWT create/verify.
`src/validate.rs`	Username / ID / timestamp guards.
`src/rate_limit.rs`	In-memory, per-IP sliding-window limiter.
`src/error.rs`	`AppError` enum and its mapping to HTTP responses.
`src/ws.rs`	The `WsConnections` registry (who is connected, and how to reach them).
`src/routes/*.rs`	One module per resource group: auth, users, channels, messages, attachments, and the WebSocket handler.

2.3 `AppState`: The Shared Handle

Every handler receives a cloned `AppState` via Axum's `State` extractor. It is cheap to clone — everything inside is either an `Arc` or a LanceDB `Connection` (itself internally reference-counted). It is defined in `src/lib.rs`:

```
#[derive(Clone)]
pub struct AppState {
    pub cfg: Arc<Config>,
    pub conn: lancedb::connection::Connection,
    pub rate_limiter: Arc<RateLimiter>,
    pub ws_connections: Arc<WsConnections>,
}
```

- `cfg` — the immutable, fully-resolved configuration.
- `conn` — the single shared LanceDB connection; tables are opened on demand through `db_ops::open`.
- `rate_limiter` — the shared abuse limiter (see §7).
- `ws_connections` — the live WebSocket registry used to relay messages to connected recipients.

2.4 The Router

`build_router` in `src/lib.rs` wires every path to its handler, attaches a `TraceLayer` for request logging, and injects the state. The complete route table:

Method	Path	Handler
GET	`/`	embedded `index.html`
POST	`/api/register`	`routes::auth::register`

Method	Path	Handler
POST	`/api/login`	`routes::auth::login`
GET	`/api/users`	`routes::users::list_users`
GET	`/api/users/search`	`routes::users::search_users`
GET	`/api/users/{target}/public_key`	`routes::users::get_public_key`
GET/POST	`/api/channels`	`list_channels` / `create_channel`
GET	`/api/channels/browse`	`routes::channels::browse_channels`
GET	`/api/channels/{id}`	`routes::channels::get_channel`
POST	`/api/channels/{id}/join`	`routes::channels::join_channel`
POST	`/api/channels/{id}/members`	`routes::channels::add_channel_member`
GET	`/api/channels/{id}/members/public_keys`	`get_channel_member_keys`
GET	`/api/messages/{peer}`	`routes::messages::get_dm_messages`
GET/POST	`/api/channels/{id}/messages`	`get_channel_messages` / `post_channel_message`
GET	`/api/attachments/{id}`	`routes::attachments::get_attachment`
GET	`/ws/chat`	`routes::ws::ws_chat`
GET	`/static/{*path}`	embedded static assets

2.5 Lifecycle of a Request

The bootstrap sequence in `src/main.rs` is deliberately linear:

```
let cfg = Arc::new(Config::load()?);        // 1. resolve configuration
let conn = db::connect(&cfg.database_path).await?;  // 2. open LanceDB
db::init_db(&conn).await?;                   // 3. create/migrate tables
let state = AppState { cfg, conn, rate_limiter, ws_connections };
let app = build_router(state);               // 4. wire routes
let listener = TcpListener::bind(addr).await?;
axum::serve(listener, app.into_make_service_with_connect_info::<SocketAddr>()).await
    ?;
```

Note `into_make_service_with_connect_info::<SocketAddr>()`: it exposes the peer socket address to handlers, which the rate limiter needs to key on the client IP.

A typical authenticated REST request then flows:

1. Axum extracts `State<AppState>`, path/query params, and (for writes) a JSON body.
2. The handler optionally calls `rate_limiter.check(ip)` (auth and search endpoints).
3. It authenticates by calling `require_auth(&cfg, &token)`, turning the query-string token into a username or a 401.
4. It validates inputs (`validate_username`, `validate_id`, `validate_after`).
5. It authorizes (e.g. channel-membership checks).
6. It reads/writes LanceDB through `db_ops` helpers.
7. It returns `Json<T>` on success or an `AppError` that `IntoResponse` renders as `{"detail": "..."}` with the right status.

The WebSocket path (`/ws/chat`) is different: it upgrades the connection, registers it in `WsConnections`, and then loops on inbound frames, storing each message and relaying it to connected recipients. That flow is the subject of Chapter 6.

Chapter 3

Configuration

All configuration lives in `src/config.rs`. The public product is a single immutable `Config` struct, resolved once at startup and shared through `AppState` as `Arc<Config>`.

3.1 The Three-Layer Cascade

Values are resolved in ascending order of precedence:

1. **Embedded defaults.** The repository's `config.yaml` is baked into the binary at compile time:

   ```
   const EMBEDDED_CONFIG_YAML: &str =
       include_str!(concat!(env!("CARGO_MANIFEST_DIR"), "/config.yaml"));
   ```

   This guarantees the binary is runnable with no external files.

2. **On-disk `config.yaml`.** If a `config.yaml` exists at the project root, it is read *instead of* the embedded copy (it wholly replaces it; the two are not merged). See `read_yaml_or_embedded`.

3. **Environment variables.** Individual fields are then overridden by env vars, which always win.

3.2 The YAML Shape

The default `config.yaml`:

```
app:
  title: "Seal"
  host: "0.0.0.0"
  port: 8000

database:
  path: "data/chat.lance"

auth:
  jwt_algorithm: "HS256"
  token_expire_minutes: 1440  # 24 hours

validation:
  username_max_length: 64
```

```
  id_max_length: 128

attachments:
  max_image_size_mb: 5
```

These map to the private `YamlConfig` deserialization structs (`AppSection`, `DatabaseSection`, `AuthSection`, `ValidationSection`, `AttachmentsSection`). The `attachments` section is `#[serde(default)]` with a per-field default of 5 MB (`default_image_size_mb`), so older config files without it still parse.

3.3 Environment Variables

Variable	Overrides	Notes
`JWT_SECRET`	(required)	HMAC signing secret. If unset, defaults to `"change-me"` and logs a warning — development only.
`APP_TITLE`	`app.title`	
`APP_HOST`	`app.host`	
`APP_PORT`	`app.port`	Parsed to `u16`; an unparseable value fails startup.
`DATABASE_PATH`	`database.path`	Relative → anchored to project root; absolute → used verbatim.
`AUTH_JWT_ALGORITHM`	`auth.jwt_algorithm`	`HS256`, `HS384`, or `HS512`.
`AUTH_TOKEN_EXPIRE_MINUTES`	`auth.token_expire_minutes`	Parsed to `i64`.
`SEAL_PROJECT_ROOT`	(see below)	Where to look for `.env` / `config.yaml` and what to anchor relative DB paths to.

`env_or` and `env_or_parse` implement the override: the former returns the env value or a default string; the latter additionally parses to a target type and turns parse failures into a startup error rather than silently falling back.

3.4 `.env` Loading and the Project Root

`Config::load` begins by loading a `.env` file (via `dotenvy`):

```
if let Ok(dir) = std::env::var("SEAL_PROJECT_ROOT") {
    let _ = dotenvy::from_path(PathBuf::from(&dir).join(".env"));
} else {
    let _ = dotenvy::dotenv();   // walk up from CWD
}
```

A missing `.env` is not an error — the variables may already be present in the process environment (as in Docker or CI). `project_root` then honours `SEAL_PROJECT_ROOT` if set, otherwise falls back to the current working directory. This anchor determines both where an on-disk `config.yaml` is sought and how a relative `DATABASE_PATH` is resolved:

```
let database_path = PathBuf::from(env_or("DATABASE_PATH", &yaml.database.path));
let database_path = if database_path.is_absolute() {
    database_path
} else {
    project_root.join(database_path)
};
```

3.5 Derived Fields

`Config::load` does more than copy strings:

- `max_image_size_bytes` is computed as `max_image_size_mb * 1024 * 1024`.
- `safe_name_re` and `safe_id_re` are compiled once from the configured maximum lengths:

  ```
  let safe_name_re = Regex::new(&format!(
      r"^[a-zA-Z0-9_\-]{{1,{}}}$", yaml.validation.username_max_length))?;
  ```

  Compiling the regexes here (rather than per request) means every validation call is a cheap match against a prebuilt automaton. See Chapter 7 for how they are used.
- The `jwt_secret` default emits a `tracing::warn!` so a production deployment that forgets to set it is at least noisy.

3.6 `Config::for_test`

A second constructor exists purely for tests:

```
pub fn for_test(database_path: PathBuf, jwt_secret: String) -> anyhow::Result<Self>
```

It builds a `Config` from the YAML but takes the database path and JWT secret as explicit arguments and *ignores all environment variables*. Because Rust's test harness runs tests in parallel within one process, reading process-global env state would race; `for_test` sidesteps that entirely. This is the constructor the integration harness in `tests/common/mod.rs` uses.

3.7 Configuration Unit Tests

`src/config.rs` carries its own `#[cfg(test)]` module that serializes env-mutating tests behind a `Mutex` and snapshots/restores the relevant variables around each case. It verifies: `project_root` precedence, the embedded-defaults path, env overrides (including that an absolute `DATABASE_PATH` is used verbatim), and that an unparseable `APP_PORT` fails `load()`.

Chapter 4

The Database Layer

Seal persists to **LanceDB**, an embedded database built on Apache Arrow. There is no SQL server and no ORM. Data is modelled as Arrow `RecordBatch`es against fixed schemas, and queries are expressed as filter predicates over columns. Two files carry this layer: `src/db.rs` (schemas, connection, initialization, migration) and `src/db_ops.rs` (the ergonomic helpers used by route handlers).

4.1 The Five Tables

Every schema is defined in `src/db.rs` as a function returning a `SchemaRef`. All fields are nullable (`true`). Types are Arrow types: `Utf8` (string), `LargeUtf8` (large string, for big blobs), and `Float64` (used for timestamps and, thus, sub-second precision).

`users`

Column	Type	Meaning
`username`	Utf8	Unique handle (validated on write).
`password_hash`	Utf8	bcrypt hash.
`public_key_jwk`	Utf8	The user's public key, as a JWK string.

`messages`

The busiest table. A single logical message becomes one row per recipient copy.

Column	Type	Meaning
`id`	Utf8	UUID of this row.
`sender`	Utf8	Author username.
`recipient`	Utf8	Target username.
`channel_id`	Utf8	Channel UUID, or `''` for a received DM, or `'self'` for a DM self-copy.
`ciphertext`	Utf8	libsodium ciphertext (base64/JWK-ish string).
`iv`	Utf8	Nonce / IV.
`sender_public_key_jwk`	Utf8	Ephemeral sender public key for this row.
`timestamp`	Float64	Unix seconds, fractional.
`message_type`	Utf8	`"text"` or `"image"`.
`attachment_id`	Utf8	Attachment UUID for images, else `''`.

The `channel_id` column is overloaded and is the key to understanding DM storage (Chapter 6): a received DM is stored with an empty `channel_id`, while the sender's own decryptable copy is stored with `channel_id = 'self'`.

`attachments`

Column	Type	Meaning
`id`	Utf8	UUID.
`sender`	Utf8	Uploader.
`channel_id`	Utf8	Owning channel (drives the access check).
`encrypted_data`	LargeUtf8	The encrypted image payload.
`iv`	Utf8	Nonce / IV.
`timestamp`	Float64	Unix seconds.

`encrypted_data` is `LargeUtf8` specifically because image ciphertext can exceed the 2 GiB offset limit of the ordinary `Utf8` array type.

`channels` and `channel_members`

Table.Column	Type	Meaning
`channels.id`	Utf8	Channel UUID.
`channels.name`	Utf8	Unique, human name.
`channels.created_by`	Utf8	Creator username.
`channels.created_at`	Float64	Unix seconds.
`channel_members.channel_id`	Utf8	FK to `channels.id`.
`channel_members.username`	Utf8	Member.
`channel_members.joined_at`	Float64	Unix seconds.

Membership is a plain join table; there is no uniqueness constraint enforced by the store, so handlers check for duplicates in application code before inserting.

4.2 Connecting and Initializing

`connect` opens (or creates) the database at a filesystem path:

```
pub async fn connect(path: &std::path::Path) -> anyhow::Result<Connection> {
    if let Some(parent) = path.parent() {
        std::fs::create_dir_all(parent).ok();   // ensure the directory exists
    }
    let path_str = path.to_str()
        .ok_or_else(|| anyhow::anyhow!("non-UTF8 database path: {}", path.display())
    )?;
    let conn = lancedb::connect(path_str).execute().await?;
    Ok(conn)
}
```

> **Branch note.** On `main` this signature is filesystem-only. The `feat/object-storage` branch extends it to accept storage options and to skip directory creation for `s3://`, `gs://`, `az://` URIs. See Chapter 11.

`init_db` is idempotent. It lists existing tables and creates any that are missing with `create_empty_table`. Crucially, if the `messages` table already exists, it runs a migration

instead of skipping:

```
if !existing.contains("messages") {
    conn.create_empty_table("messages", messages_schema()).execute().await?;
} else {
    migrate_messages_table(conn).await?;
}
```

4.3 The Messages Migration

`migrate_messages_table` mirrors an earlier Python migration. It inspects the live schema and, if the `message_type` or `attachment_id` columns are absent (as they would be in a database created by an older build), adds them with SQL default expressions:

```
if !field_names.contains("message_type") {
    new_columns.push(("message_type".to_string(), "'text'".to_string()));
}
if !field_names.contains("attachment_id") {
    new_columns.push(("attachment_id".to_string(), "''".to_string()));
}
// ...
tbl.add_columns(NewColumnTransform::SqlExpressions(new_columns), None).await?;
```

If both columns are already present the function returns early, so re-running it is a no-op — a property the migration tests assert directly.

4.4 `db_ops`: The Query Toolkit

Route code never touches LanceDB's Arrow machinery directly. Instead it uses a small vocabulary of helpers in `src/db_ops.rs`, each of which maps LanceDB errors into `AppError::Internal`:

Helper	Purpose
`open(conn, name)`	Open a table by name.
`scan_where(tbl, pred, limit)`	Run a filtered scan (`only_if(pred)`) and collect all matching `RecordBatch`es, optionally capped by `limit`.
`append(tbl, batch)`	Append a one-batch set of rows.
`first_string(batches, col)`	First value of a `Utf8` column from the first non-empty batch (null → empty string, missing → `None`).
`first_large_string(batches, col)`	Same, for `LargeUtf8` columns.
`collect_string_column(batches, col)`	Every non-null value of a `Utf8` column across all batches.
`total_rows(batches)`	Sum of row counts — the "does anything match?" primitive.
`utf8_row(schema, fields)`	Build a single all-`Utf8` row from `(name, value)` pairs, matched to the schema by field name.
`mixed_row(schema, cells)`	Build a single row from an ordered slice of `Cell::{Str, LargeStr, F64}` values.
`rows_to_stored_messages(batches)`	Materialize `messages`-table rows into the API's `StoredMessage` shape.

The `Cell` enum is the trick that lets `mixed_row` build heterogeneous rows (strings, large strings, and `f64` timestamps) without bespoke builder code at each call site:

```
pub enum Cell<'a> { Str(&'a str), LargeStr(&'a str), F64(f64) }
```

`mixed_row` validates that the number of cells matches the schema's field count and returns `AppError::Internal` otherwise — a guard the unit tests exercise. `db_ops.rs` carries an extensive `#[cfg(test)]` module that covers row building, null handling, wrong-type handling, and the `StoredMessage` materializer.

4.5 A Word on Query Safety

Predicates are built with `format!` string interpolation, e.g. `format!("username = '{username}'")`. This is safe here *because* every interpolated identifier is first passed through the validation guards (§7), which restrict usernames and IDs to `[A-Za-z0-9_-]`. No quote, space, or SQL metacharacter can reach a predicate, so there is no injection surface. New handlers must preserve this invariant: validate before you interpolate.

Chapter 5

The REST API

This chapter is the endpoint reference. Handlers live under `src/routes/`; request/response bodies are the `serde` structs in `src/models.rs`.

5.1 Conventions

- **Authentication is by query-string token.** Protected endpoints read `?token=<jwt>` (modelled by `TokenQuery`, `SearchQuery`, or `AfterQuery`) and call `require_auth`, which returns the username or a `401 {"detail":"Invalid token"}`. This mirrors the original Python server and lets the same token authorize the WebSocket upgrade, which cannot carry custom headers from a browser.
- **Errors share one shape.** Every failure is `{"detail": "<message>"}` with an appropriate status. See Chapter 7 and `src/error.rs`.
- **Status codes.** `200` success, `400` bad request, `401` unauthorized, `403` forbidden, `404` not found, `429` rate-limited, `500` masked internal error. (JSON body validation failures surface as Axum's `422`.)

5.2 Authentication — `src/routes/auth.rs`

POST `/api/register`

Rate-limited. Body `RegisterRequest`: `username`, `password`, `public_key_jwk`. The handler validates the username, rejects a duplicate (`"Username already taken"`), bcrypt-hashes the password, inserts the user row, and returns a fresh token:

```
{ "token": "<jwt>", "username": "alice" }   // TokenResponse
```

POST `/api/login`

Rate-limited. Body `LoginRequest`: `username`, `password`. Looks up the user, verifies the password against the stored bcrypt hash, and returns a `TokenResponse`. A missing user and a wrong password are deliberately indistinguishable: both yield `401 "Invalid credentials"`.

5.3 Users — `src/routes/users.rs`

GET `/api/users`

Returns the caller's DM contacts — everyone they have exchanged direct messages with. It derives this from the `messages` table rather than a contacts table: the union of recipients of the caller's `'self'` copies and senders of DMs addressed to the caller.

```
sent     = messages WHERE sender = <me>    AND channel_id = 'self'
received = messages WHERE recipient = <me> AND channel_id = ''
peers    = sorted-unique( sent.recipient  U  received.sender )
```

Response: a JSON array of `{ "username": ... }` (`UserListItem`).

GET `/api/users/search?q=<prefix>`

Rate-limited. Prefix-matches usernames (`username LIKE '<q>%'`), excludes the caller, and truncates to 20 results. `q` is required and is itself validated as a username so the `LIKE` pattern cannot contain metacharacters.

GET `/api/users/{target}/public_key`

Returns `{ "username", "public_key_jwk" }` (`UserPublicKeyResponse`) so a client can encrypt to `target`. `404 "User not found"` if absent.

5.4 Channels — `src/routes/channels.rs`

POST `/api/channels`

Body `CreateChannelRequest`: `name` and optional `members`. The handler validates the name (same charset/length as a username) and every named member, **rejects a duplicate channel name**, then verifies that the creator and every named member actually exist in `users`. It assigns a UUID, writes the `channels` row, and inserts all membership rows in a single batch (the creator is always included and de-duplicated). Returns `ChannelInfo`:

```
{ "id", "name", "created_by", "members": ["alice", "bob", ...] }
```

GET `/api/channels`

Lists channels the caller belongs to. It reads the caller's `channel_members` rows, then fetches each channel and its full member list, returning an array of `ChannelInfo`.

GET `/api/channels/browse`

The inverse: channels the caller is *not* in. It scans all channels, skips those in the caller's membership set, and returns `ChannelBrowseItem` objects that carry a `member_count` instead of the full roster:

```
{ "id", "name", "created_by", "member_count": 7 }
```

POST `/api/channels/{id}/join`

Adds the caller to a channel. `404` if the channel does not exist, `400 "Already a member"` if they are in it. On success returns the updated `ChannelInfo`.

GET `/api/channels/{id}`

Returns `ChannelInfo` *only* to members; non-members get `403 "Not a member of this channel"`.

POST `/api/channels/{id}/members`

Body `AddChannelMemberRequest`: `username`. The caller must be a member (`403` otherwise); the target must exist (`404`) and not already be a member (`400`). On success returns `{"status": "ok"}`.

GET `/api/channels/{id}/members/public_keys`

Members-only. Returns an array of `ChannelMemberKey` (`{ "username", "public_key_jwk" }`) — exactly what a client needs to encrypt one envelope per recipient before posting a channel message.

5.5 Messages — `src/routes/messages.rs`

GET `/api/messages/{peer}?after=<ts>`

Direct-message history with one peer. Reads two slices of the `messages` table and merges them, sorted by timestamp:

```
sent-self = WHERE sender=<me>   AND recipient=<peer> AND channel_id='self'
received  = WHERE sender=<peer> AND recipient=<me>   AND channel_id=''
```

The optional `after` (a float Unix timestamp, validated by `validate_after`) appends `AND timestamp > <after>` to both queries for incremental polling. Each row is a `StoredMessage`.

GET `/api/channels/{id}/messages?after=<ts>`

Channel history for the caller. Members-only. Returns only the rows encrypted *for the caller* (`channel_id = <id> AND recipient = <me>`), sorted by timestamp — the fan-out storage model (Chapter 7) means each member reads their own copies.

POST `/api/channels/{id}/messages`

The REST alternative to sending over the WebSocket. Body `ChannelMessagePayload`:

```
{
  "channel_id": "<uuid>",
  "envelopes": [
    { "target_user", "ciphertext", "iv", "sender_public_key_jwk" }, ...
  ],
  "message_type": "text" | "image",     // default "text"
  "attachment": { "encrypted_data", "iv" }   // optional, images only
}
```

Members-only. It calls the shared `store_channel_message` (one row per envelope; an image also writes one `attachments` row) and then `relay_channel_message` to push a live frame to any connected recipient. Returns `{ "status":"ok", "group_id", "timestamp" }`. This handler and the WebSocket channel handler share the very same persistence and relay functions.

5.6 Attachments — `src/routes/attachments.rs`

GET `/api/attachments/{id}`

Fetches an encrypted attachment blob. If the attachment belongs to a channel, the caller must be a member of that channel (`403` otherwise); attachments with an empty `channel_id` skip that check. `404` if the id is unknown. Returns `AttachmentResponse`: `{ "id", "encrypted_data", "iv" }`. As everywhere, `encrypted_data` is ciphertext — the server cannot render the image.

Chapter 6

Real-Time Messaging over WebSocket

The live path is `GET /ws/chat?token=<jwt>`, handled by `src/routes/ws.rs`, backed by the connection registry in `src/ws.rs`. REST endpoints persist and read history; the WebSocket is how a message reaches a recipient *now*.

6.1 The Connection Registry

`WsConnections` (`src/ws.rs`) is the in-process map of who is online and how to reach them:

```
pub struct WsConnections {
    inner: Mutex<HashMap<String, Vec<(Uuid, ConnTx)>>>,
}
pub type ConnTx = UnboundedSender<Message>;
```

A user maps to a *list* of connections because one account can be signed in from several devices/tabs at once. Each entry pairs a per-connection `Uuid` with an unbounded mpsc sender that feeds that socket's writer task. The API:

Method	Effect
`register(user, id, tx)`	Add a connection for `user`.
`unregister(user, id)`	Remove it; drop the user key entirely if it was the last connection.
`send_to(user, payload)`	Send text to *all* of `user`'s connections.
`send_to_except(user, id, payload)`	Send to all but the connection `id` — used to echo a DM to the sender's *other* devices without looping it back to the origin.

Sends are best-effort: `let _ = tx.send(...)`. A dead receiver simply drops the frame; the connection is cleaned up when its receive loop ends. The registry has a thorough unit-test module (multi-connection delivery, the "except" exclusion, no-op sends to unknown users, and last-connection removal).

6.2 Upgrade and the Per-Connection Tasks

`ws_chat` decodes the token *before* upgrading. An invalid token still upgrades but is immediately closed with code `4001 "Invalid token"` — the handshake completes so the client gets a clean, typed rejection rather than a bare HTTP error.

On a valid token, `handle_socket` sets up the machinery:

1. Mint a `conn_id: Uuid` for this socket.
2. Split the socket into a `sink` (outbound) and `stream` (inbound).
3. Create an unbounded mpsc channel and spawn a **writer task** that drains the channel into the sink. All outbound frames — relays, acks, errors — go through this one channel, so any handler (even one running for a *different* user) can enqueue a frame for this socket without sharing the sink.
4. `register` the `(conn_id, tx)` under the username.
5. Loop over inbound frames until the stream ends or a close frame arrives.
6. On exit, `unregister`, drop the sender, and await the writer task.

6.3 Frame Dispatch

Each inbound text frame is parsed as JSON and dispatched on its `"type"` field, defaulting to `"dm"`:

```
match data.get("type").and_then(|v| v.as_str()).unwrap_or("dm") {
    "channel" => handle_channel(...).await,
    _         => handle_dm(...).await,
}
```

Malformed JSON is logged and skipped, not fatal — a bad frame never tears down the connection.

6.4 Direct Messages: the Double-Write

`handle_dm` is where the `channel_id` overloading from Chapter 4 pays off. A DM inbound frame looks like:

```
{
  "type": "dm",
  "recipient": "bob",
  "ciphertext": "...", "iv": "...", "sender_public_key_jwk": "...",
  "self_ciphertext": "...", "self_iv": "...", "self_sender_public_key_jwk": "..."
}
```

The client encrypts the message *twice*: once to the recipient's public key, once to its own. The handler therefore writes up to two rows:

1. **The recipient copy** — `recipient = bob`, `channel_id = ''`, using `ciphertext`/`iv`/`spk`. This is what Bob reads back via `GET /api/messages/alice`.
2. **The self copy** — only if all three `self_*` fields are present — with `channel_id = 'self'`, using the `self_*` values. This is what Alice reads back so she can decrypt her own sent

history. (Because the recipient copy is encrypted to Bob's key, Alice *cannot* decrypt it; she needs her own copy.)

Both rows share the same timestamp. The message id returned to the sender is the recipient copy's id.

Relay then happens in three directions:

```
state.ws_connections.send_to(recipient, &relay_text);          // to Bob
state.ws_connections.send_to_except(sender, own_id, &relay_text); // Alice's OTHER
    devices
own_tx.send(ack);                                              // ack to THIS socket
```

The relayed frame is a `"dm"` object carrying the id, sender, recipient, ciphertext, iv, sender public key, timestamp, and message type. The ack is the same frame plus an `"ack"` field holding the message id.

6.5 Channel Messages: Fan-Out

`handle_channel` parses the frame into the very same `ChannelMessagePayload` the REST handler uses. It verifies the sender is a member of `payload.channel_id` (replying with an `{"error": ...}` frame if not), then delegates to the shared functions in `src/routes/messages.rs`:

- `store_channel_message` — writes one `messages` row per envelope (each encrypted to a different member) plus, for an image, one `attachments` row. It returns a `ChannelMessageStoreResult` with a `group_id` (shared logical-message id), the timestamp, the attachment id, the resolved message type, and the per-envelope row ids.
- `relay_channel_message` — iterates envelopes zipped with their row ids and `send_to` each `target_user`, delivering each member their own ciphertext.

The sender receives an ack keyed by `group_id`. Because storage and relay are shared functions, a channel message sent over REST and one sent over the WebSocket are byte-for-byte equivalent in the database and to recipients — the transport is an implementation detail.

6.6 Why This Shape

Storing a separate ciphertext per recipient (fan-out) is what keeps the server zero-knowledge for groups: there is no group key the server could hold. The cost is storage and write amplification proportional to channel size; the benefit is that membership changes and per-recipient delivery need no re-keying on the server, and the server literally cannot read the traffic it carries.

Chapter 7

Encryption and the Security Model

Seal's security rests on a simple division of labour: **the client does the cryptography; the server does the bookkeeping**. This chapter covers both — the end-to-end encryption scheme and the server-side defences (authentication, authorization, rate limiting, input validation, and error hygiene).

7.1 End-to-End Encryption

Keys

Each user has an X25519 key pair generated on the client with libsodium. The public half is uploaded at registration (`public_key_jwk`) and served to peers via `GET /api/users/{target}/public_key` and, for groups, `GET /api/channels/{id}/members/public_keys`. The private half never leaves the device.

Encrypting a message

To message a peer, the client fetches the peer's public key and seals the plaintext with libsodium (a `crypto_box`-style operation: X25519 key agreement plus an authenticated stream cipher). The wire carries only the `ciphertext`, the `iv`/nonce, and an ephemeral `sender_public_key_jwk`. The server stores these opaque strings verbatim.

DMs: encrypt twice

Because a message sealed to Bob's key is unreadable by Alice, the sender encrypts a *second* copy to her own key. The server stores the recipient copy (`channel_id = ''`) and the self copy (`channel_id = 'self'`), as detailed in Chapter 6. History reconstruction merges the two.

Channels: encrypt per member (fan-out)

There is no shared group key. To post to a channel the client encrypts one *envelope* per member, each sealed to that member's public key, and submits them together. The server stores one row per envelope and delivers each member their own ciphertext. A channel of n members yields n stored ciphertexts per message.

What the server can and cannot see

Visible to the server	Never visible
Usernames and public keys	Private keys
Who messaged whom, and when	Message plaintext
Channel membership	Image contents
Ciphertext, IVs, sizes	Anything decryptable

A full database compromise therefore leaks the social graph and ciphertext, but not message contents. Seal does not claim to hide metadata — only content.

7.2 Passwords: bcrypt

`src/auth.rs` hashes passwords with bcrypt at the library's `DEFAULT_COST`:

```
pub fn hash_password(plain: &str) -> Result<String, AppError> {
    bcrypt::hash(plain, DEFAULT_COST)
        .map_err(|e| AppError::Internal(anyhow::anyhow!("bcrypt hash failed: {e}")))
}
pub fn verify_password(plain: &str, hashed: &str) -> bool {
    bcrypt::verify(plain, hashed).unwrap_or(false)
}
```

`verify_password` returns `false` on *any* error, including a malformed stored hash — it never panics and never treats an unparseable hash as a match. The `$2b$` bcrypt format is wire-compatible with the Python predecessor, so hashes created by the old server still verify.

7.3 Sessions: JSON Web Tokens

Tokens are HMAC-signed JWTs. `create_token` stamps a `sub` (username) and an `exp` computed from `token_expire_minutes` (clamped at zero so a negative config cannot backdate). `algorithm_from` accepts only `HS256`, `HS384`, and `HS512`; anything else is an internal error.

Verification is deliberately forgiving in *form* but strict in *outcome*:

```
pub fn require_auth(cfg: &Config, token: &str) -> Result<String, AppError> {
    decode_token(cfg, token).ok_or(AppError::Unauthorized("Invalid token"))
}
pub fn decode_token(cfg: &Config, token: &str) -> Option<String> {
    let validation = Validation::new(algorithm_from(cfg).ok()?);
    decode::<Claims>(token, &DecodingKey::from_secret(cfg.jwt_secret.as_bytes()),
                     &validation).ok().map(|d| d.claims.sub)
}
```

Any failure — wrong secret, garbage input, or an expired token (the default 60-second leeway aside) — collapses to `None` and thus a single `401 "Invalid token"`. The unit tests cover round-trips for all three algorithms, wrong-secret rejection, garbage input, and expiry.

7.4 Rate Limiting

`src/rate_limit.rs` implements an in-memory sliding window, keyed by the direct peer IP:

```
const RATE_LIMIT_WINDOW_SECS: f64 = 60.0;
const RATE_LIMIT_MAX: usize = 20;
```

`check(ip)` prunes timestamps older than 60 seconds, rejects with `AppError::TooManyRequests` (`429`) if 20 are already in the window, and otherwise records `now`. It is applied to the abuse-prone endpoints: `/api/register`, `/api/login`, and `/api/users/search`.

Two caveats worth knowing when operating Seal:

- The limiter keys on the socket peer IP (via `ConnectInfo<SocketAddr>`). Behind a reverse proxy, that is the proxy's IP unless the proxy is configured to preserve the client address; the limiter does not read `X-Forwarded-For`.
- State is per-process and in-memory: it resets on restart and is not shared across horizontally-scaled instances.

7.5 Input Validation

`src/validate.rs` holds the guards, all backed by the regexes compiled in `Config`:

- `validate_username` — must match `^[A-Za-z0-9_-]{1,64}$` (length from config). Error message: "Username must be 1-64 alphanumeric characters, hyphens, or underscores".
- `validate_id` — same charset, up to the configured id length (128). Used for channel and attachment ids.
- `validate_after` — rejects NaN, infinity, and negatives so the polling cursor cannot smuggle a malformed float into a query.

These guards are the linchpin of query safety (§4): because every identifier interpolated into a LanceDB predicate has first passed a guard restricting it to `[A-Za-z0-9_-]`, no quote or metacharacter can reach the query string. *Validate before you interpolate* is the rule any new handler must follow.

7.6 Authorization

Beyond authentication, handlers enforce resource-level rules in application code:

- Channel reads and membership changes require the caller to be a member (`is_channel_member` / `assert_channel_member` → `403 "Not a member of this channel"`).
- Attachment reads require channel membership when the attachment belongs to a channel.
- DM history is scoped to the caller by construction of the query predicates.

7.7 Error Hygiene

`src/error.rs` defines `AppError` and its `IntoResponse`. Client errors pass their message through; the `Internal` variant is different:

```
AppError::Internal(e) => {
    tracing::error!("internal error: {e:?}");   // full detail to the log
    (StatusCode::INTERNAL_SERVER_ERROR, "Internal server error".to_string())
}
```

The underlying `anyhow` chain — which may mention database paths or internal state — is logged but *never* returned to the client, which sees only a generic 500. A dedicated test asserts that a secret in the error does not leak into the body. The `#[from] anyhow::Error` on `Internal` lets any `?` in a handler funnel unexpected failures into this masked path.

Chapter 8

Source Reference

A module-by-module tour of `src/`. Earlier chapters explained the *why*; this is the *where* — the map you use when editing a specific file. Public items are named so you can grep for them.

8.1 `src/main.rs`

The binary entry point. Initializes `tracing_subscriber` with an env filter defaulting to `info,tower_http=info`, loads the config, connects and initializes the database, assembles `AppState`, builds the router, binds a `TcpListener`, and calls `axum::serve` with connect-info so peer IPs reach handlers. It is intentionally thin and is the one module not covered by unit tests (its logic is exercised through the integration harness instead).

8.2 `src/lib.rs`

The library root. It:

- declares every module (`auth`, `config`, `db`, `db_ops`, `error`, `models`, `rate_limit`, `routes`, `validate`, `ws`);
- embeds `static/` and `templates/` via `include_dir!`;
- defines `AppState` (§2);
- defines `build_router`, the single source of truth for the route table;
- serves `index` (the embedded `index.html`) and `serve_static` (embedded assets with a `mime_guess` content type), plus an `internal_error` helper that logs and returns a plain 500.

8.3 `src/config.rs`

Layered configuration; see Chapter 3. Public surface: `Config` (the resolved struct), `Config::load`, and `Config::for_test`. Private helpers: `read_yaml_or_embedded`, `project_root`, `env_or`, `env_or_parse`, and the `YamlConfig` deserialization structs. Carries a `#[cfg(test)]` module for the cascade.

8.4 src/db.rs

Schemas and lifecycle; see Chapter 4. Public: the five schema functions (`users_schema`, `messages_schema`, `attachments_schema`, `channels_schema`, `channel_members_schema`), `connect`, and `init_db`. Private: `migrate_messages_table`.

8.5 src/db_ops.rs

The query toolkit (§4). Public: `open`, `scan_where`, `append`, `first_string`, `first_large_string`, `collect_string_column`, `total_rows`, `utf8_row`, `mixed_row` (with the `Cell` enum), and `rows_to_stored_messages`. Extensively unit-tested.

8.6 src/models.rs

The serde DTOs. Requests (`Deserialize`): `RegisterRequest`, `LoginRequest`, `CreateChannelRequest`, `AddChannelMemberRequest`, `ChannelEncryptedEnvelope`, `ChannelAttachment`, `ChannelMessagePayload`. Query strings: `TokenQuery`, `SearchQuery`, `AfterQuery`. Responses (`Serialize`): `TokenResponse`, `UserListItem`, `UserPublicKeyResponse`, `ChannelInfo`, `ChannelBrowseItem`, `ChannelMemberKey`, `StoredMessage`, `AttachmentResponse`. Note the `#[serde(default)]` attributes: channel `members` default to empty, `after` to `0.0`, and `message_type` to `"text"` via `default_message_type`.

8.7 src/auth.rs

bcrypt and JWT; see Chapter 7. Public: `hash_password`, `verify_password`, `create_token`, `require_auth`, `decode_token`. Private: the `Claims` struct and `algorithm_from`. Unit tests cover the algorithm mapping, password round-trips, token round-trips, wrong-secret and garbage rejection, and expiry.

8.8 src/validate.rs

The input guards `validate_username`, `validate_id`, and `validate_after` (§7). `validate_id` and `validate_after` are marked `#[allow(dead_code)]` at the definition site but are used by the channel/message handlers; the attribute simply suppresses a false-positive lint. Unit-tested for accepted and rejected inputs.

8.9 src/rate_limit.rs

The `RateLimiter` sliding window (§7): `new`, `check`, and a test-only `clear`. The window (60 s) and cap (20) are module constants. Unit-tested for the limit boundary, per-IP isolation, and window reset.

8.10 src/error.rs

`AppError` and its HTTP mapping (§7). Variants: `BadRequest(String)`, `Unauthorized(&'static str)`, `Forbidden(&'static str)`, `NotFound(String)`,

`TooManyRequests`, and `Internal(#[from] anyhow::Error)`. `AppResult<T>` is the handler return alias. `IntoResponse` renders `{"detail": ...}` with the right status and masks internals. Async unit tests assert each variant's status/body and the masking of internal errors.

8.11 src/ws.rs

The `WsConnections` registry (§6): `register`, `unregister`, `send_to`, `send_to_except`, over a `Mutex<HashMap<String, Vec<(Uuid, ConnTx)>>>`. Unit-tested for fan-out to multiple connections, the exclusion path, no-op sends, and last-connection removal.

8.12 src/routes/

Handlers, one module per resource. `mod.rs` declares them.

auth.rs

`register` and `login` — rate-limited, validated, bcrypt/JWT-backed (Chapter 5).

users.rs

`list_users` (DM peers derived from the messages table), `search_users` (rate-limited prefix search, capped at 20), and `get_public_key`.

channels.rs

The largest handler module. Public handlers: `create_channel`, `list_channels`, `browse_channels`, `join_channel`, `get_channel`, `add_channel_member`, `get_channel_member_keys`. Private helpers worth knowing: `list_channel_member_usernames`, `is_channel_member`, `fetch_channel` (returns a private `ChannelRow`), `channel_info_for`, `build_member_batch` (inserts all members in one `RecordBatch`), and `now_secs`.

messages.rs

`get_dm_messages`, `get_channel_messages`, and `post_channel_message`. The shared engine lives here too: `store_channel_message` (persist one row per envelope + optional attachment, returning `ChannelMessageStoreResult`) and `relay_channel_message` (push one live frame per envelope). Both are called by the REST handler and by the WebSocket channel handler — the single point that guarantees the two transports behave identically. `assert_channel_member` enforces membership.

attachments.rs

`get_attachment` — fetch an encrypted blob with a channel-membership check when the attachment is owned by a channel.

ws.rs

The WebSocket handler (Chapter 6): `ws_chat` (upgrade + token gate), `handle_socket` (registry + writer task + dispatch loop), `handle_dm` (the double-write and three-way relay), and `handle_channel` (membership check, then the shared store/relay). `text_msg` wraps a `serde_json::Value` into a WS text frame.

8.13 Cross-Cutting Idioms

A handful of patterns recur and are worth internalizing before making changes:

- **Thin handlers.** Validate → authenticate → authorize → call `db_ops`/`ws` → map errors. Business logic that two handlers share is factored out (as with the channel store/relay pair).
- **Timestamps are `f64` Unix seconds**, produced by a local `now_secs()` in each module that needs it.
- **Errors flow through `AppError`**; use `?` freely and let unexpected failures become masked 500s.
- **Never interpolate an unvalidated identifier** into a predicate.

Chapter 9

Testing

Seal is tested at two levels: **unit tests** co-located in `src/` modules, and **integration tests** in `tests/` that drive the real server over HTTP and WebSocket. At the time of writing the `tests/` tree holds 72 `#[tokio::test]` integration cases, plus unit-test modules in seven source files (`auth`, `config`, `db_ops`, `error`, `rate_limit`, `validate`, `ws`).

> **Doc drift note.** The `README` still advertises "62 native Rust integration tests"; the actual count has since grown. Treat the source tree, not the prose, as authoritative.

9.1 The Integration Harness

`tests/common/mod.rs` defines `TestServer`, which spins up the genuine application stack against a throwaway database:

```
let tempdir = tempfile::tempdir()?;                 // isolated, auto-cleaned
let db_path = tempdir.path().join("test.lance");
let cfg = Config::for_test(db_path, "test-secret-key-for-tests".into())?;
let conn = db::connect(&cfg.database_path).await?;
db::init_db(&conn).await?;
let state = AppState { cfg: Arc::new(cfg), conn,
                       rate_limiter: ..., ws_connections: ... };
let router = build_router(state.clone())
    .into_make_service_with_connect_info::<SocketAddr>();
let listener = TcpListener::bind("127.0.0.1:0").await?;  // random free port
```

The server runs in a spawned task with graceful shutdown wired to a `oneshot` channel held in the struct; when the `TestServer` drops, the server stops and the `TempDir` is deleted. Two properties make the suite robust and fast:

- **Real router, real DB.** Tests exercise `build_router` and LanceDB exactly as production does — no mocks, no in-memory substitutes. Only the config source (`for_test`) and the storage location (a temp dir) differ.
- **Parallel-safe.** Each test gets its own port, its own database, and a config that ignores process env vars, so the default parallel test runner has nothing to race on.

`TestServer` also exposes `base_url`, a `client()` builder (`reqwest`), the `state` (so a test can seed rows directly through `db_ops`), and a `url(path)` helper.

9.2 The Testing Philosophy: Seed via Internals, Assert via API

The suite's guiding pattern is to *set up* state through the library internals (writing rows with `db_ops::append`, minting tokens with `create_token`) and then to *exercise and assert* through the public HTTP/WS surface. This keeps tests fast and precise while still covering the real request path. Each test file keeps its own small local helpers (`register`, `create_channel`, `open_ws`, ...) rather than a shared "kitchen sink", so a test reads top-to-bottom.

9.3 What the Integration Files Cover

File	Focus
`tests/auth.rs`	Registration, login, duplicate-user and bad-credential paths, JWT acceptance/rejection, bcrypt compatibility, and rate limiting.
`tests/users.rs`	DM-contact derivation, prefix search (self-exclusion, the 20 cap), and public-key retrieval.
`tests/channels.rs`	Channel create/list/browse/join, member invite, membership authorization, duplicate-name and duplicate-member rejection, and member-key listing.
`tests/messages.rs`	DM history merging and `after` filtering, channel history scoping, the REST channel-send path, and attachments.
`tests/websocket.rs`	Upgrade and token gating, DM relay (including self-copy and multi-device echo), channel fan-out, and non-member rejection.
`tests/migration.rs`	The messages-table column migration and its idempotency on re-run.
`tests/static_assets.rs`	Serving the embedded `index.html` and static assets with correct content types.

9.4 Unit Tests

The co-located `#[cfg(test)]` modules test pure logic in isolation: `config` (the cascade and env precedence), `auth` (hashing, tokens, expiry), `db_ops` (row builders, null/missing/wrong-type handling, the message materializer), `error` (status/body mapping and internal masking), `rate_limit` (boundary, per-IP, reset), `validate` (accepted/rejected inputs), and `ws` (registry fan-out and cleanup).

9.5 Running the Tests

```
make test          # cargo test  -- everything
make test-quiet    # cargo test --quiet
cargo test --test websocket        # one integration file
cargo test config::                # a module's unit tests
```

9.6 Coverage and Intentionally-Untested Paths

Line coverage is measured with `cargo llvm-cov` (see the coverage CI workflow in Chapter 10). A few paths are deliberately left uncovered because exercising them would add machinery out of proportion to their risk: `src/main.rs` (the thin bootstrap, covered in spirit by the harness), the database-failure error closures inside `db_ops` (they require injecting a broken connection), and some embedded-asset "file missing / not UTF-8" branches in `lib.rs` that cannot occur with the assets that are actually compiled in. These omissions are documented rather than papered over.

Chapter 10

Building, Running, and Deploying

10.1 Dependencies at a Glance

From `Cargo.toml` (the binary is `seal-server`, Rust edition 2021):

Crate	Version	Role
`axum`	0.8	Web framework (features `ws`, `macros`).
`tokio`	1	Async runtime (`full`).
`tower` / `tower-http`	0.5 / 0.6	Middleware; `fs` + `trace`.
`serde` / `serde_json` / `serde_yml`	1 / 1 / 0.0.13	Serialization (JSON, YAML).
`dotenvy`	0.15	`.env` loading.
`bcrypt`	0.19	Password hashing.
`jsonwebtoken`	10	JWTs (`rust_crypto`).
`lancedb`	0.30	Embedded database.
`arrow-array`	58	Arrow arrays (version-locked to `lancedb`).
`uuid`	1	Ids (`v4`).
`regex`	1	Validation patterns.
`tracing` / `tracing-subscriber`	0.1 / 0.3	Logging.
`thiserror` / `anyhow`	2 / 1	Errors.
`include_dir`	0.7	Asset bundling.
`mime_guess`	2	Static-asset content types.

Dev-dependencies power the integration suite: `reqwest` (HTTP client, rustls), `tokio-tungstenite` (WebSocket client), and `tempfile` (throwaway databases).

> **Arrow / LanceDB coupling.** `arrow-array` is pinned to the major version LanceDB expects (58 for `lancedb` 0.30). Bumping one usually means bumping the other in lock-step; a mismatch produces confusing trait-resolution errors at the Arrow boundary.

10.2 The Makefile

Target	Action
`make dev` / `make build`	`cargo build` (debug).
`make release`	`cargo build --release`.
`make server`	`cargo run`.
`make test` / `make test-quiet`	Run the test suite.
`make bots`	Run the Python bot simulation against a live server.
`make clean`	`cargo clean` + remove Tauri and `__pycache__`.
`make reset-db`	Delete `data/chat.lance`.
`make bundle-sodium`	Rebuild the vendored libsodium bundle.

10.3 Running Locally

```
make dev && make server         # http://localhost:8000
# or, self-contained:
cargo run --release
```

The release binary at `target/release/seal-server` is self-contained — templates, static assets, and the default config are compiled in. To deploy it, copy the binary, set `JWT_SECRET` and `DATABASE_PATH`, and run.

10.4 Docker

The `Dockerfile` is a two-stage build:

1. **Builder** (`rust:1-bookworm`). Installs the native toolchain LanceDB needs — `protobuf-compiler`, `libprotobuf-dev`, `cmake`, `pkg-config`, `libssl-dev` — and sets `PROTOC` / `PROTOC_INCLUDE` because `lance-encoding`'s build script invokes `protoc` and imports the well-known `google/protobuf/empty.proto`. Then `cargo build --release`.
2. **Runtime** (`debian:bookworm-slim`). Copies just the binary, adds `ca-certificates`, creates `/app/data`, and sets `APP_HOST=0.0.0.0`, `APP_PORT=8080`, `DATABASE_PATH=/app/data/chat.lance`. Exposes 8080.

10.5 Continuous Integration

Two GitHub Actions workflows live in `.github/workflows/`:

- **`release.yml`** — on every push to `main` (and manually), cross-compiles the server for Linux x86_64 (`x86_64-unknown-linux-gnu`) and macOS Apple Silicon (`aarch64-apple-darwin`), packages each as a `.tar.gz`, and publishes them as a GitHub *pre-release* tagged `build-YYYYMMDD-<short-sha>`. Installs `protoc` and caches the Cargo build.
- **`coverage.yml`** — on pushes to `main`, PRs, and manual runs, uses `cargo llvm-cov` (with `llvm-tools-preview`) to produce `lcov.info`, an HTML report, and a text summary published to the workflow run.

10.6 Deploying to DigitalOcean

`scripts/deploy-digitalocean.sh` is an interactive helper for the App Platform. It prompts for the app name, GitHub repo/branch, region, and instance size, generates an app-spec YAML, and runs `doctl apps create --wait` (or `update`), optionally enabling `deploy_on_push`. Be aware that the App Platform filesystem is *ephemeral* by default: a local `DATABASE_PATH` is wiped on redeploy. For durable data, attach a persistent volume — or use the object-storage backend (Chapter 11) so LanceDB persists to a bucket.

10.7 Auxiliary Scripts and the Desktop Client

- `scripts/bots.py` — a deterministic simulation (idempotent accounts and channels, randomized messages) that drives the server with *real* PyNaCl X25519 encryption, proving cross-implementation compatibility with the JS client. Run via `make bots` (which uses `uv`).
- `scripts/bundle-sodium.sh` — rebuilds the vendored libsodium bundle shipped in `static/`.
- `desktop/` — a Tauri v2 native client (Rust + system WebView) wrapping the same web front end.

Chapter 11

Object Storage (Feature Branch)

> **Scope.** Everything in this chapter describes the `feat/object-storage` branch. It is *not* merged into `main`; the `main` database connector documented in Chapter 4 is filesystem-only. This chapter is included so the reference is complete, and each feature-branch-only element is called out as such.

11.1 Motivation

LanceDB can persist to cloud object storage (S3, GCS, Azure Blob) as easily as to a local directory — the on-disk Lance format is the same, only the `object_store` backend differs. Persisting to a bucket solves the durability problem that bites container platforms with ephemeral filesystems (§10): the data outlives any individual instance, and multiple instances can point at the same location.

11.2 URI Detection

The feature adds scheme detection so the connector can tell a bucket URI from a filesystem path:

```
const OBJECT_STORE_SCHEMES: &[&str] = &[
    "s3://", "s3a://", "gs://", "gcs://",
    "az://", "azure://", "abfs://", "abfss://",
];
pub fn is_object_store_uri(location: &str) -> bool {
    OBJECT_STORE_SCHEMES.iter().any(|s| location.starts_with(s))
}
```

11.3 The Extended Connector

`connect` gains a `storage_options` argument and two behavioural changes versus the `main` version:

```
pub async fn connect(
    location: &std::path::Path,
    storage_options: &std::collections::HashMap<String, String>,
) -> anyhow::Result<Connection> {
    // Only create local directories for filesystem paths.
    if !is_object_store_uri(path_str) {
```

```
        if let Some(parent) = location.parent() {
            std::fs::create_dir_all(parent).ok();
        }
    }
    let mut builder = lancedb::connect(path_str);
    if !storage_options.is_empty() {
        builder = builder.storage_options(storage_options.clone());
    }
    builder.execute().await
}
```

Two things change relative to `main`: object-store URIs are passed to LanceDB verbatim (no `create_dir_all`, no path joining), and credential/endpoint options are threaded through `builder.storage_options(...)`. Everything above the connector — schemas, `init_db`, the migration, every route — is byte-for-byte identical. The encryption model and data layout do not change; only where the bytes land does.

11.4 Configuration

On the feature branch, `config.yaml` grows an optional free-form `storage` section whose key/-value pairs become the LanceDB storage options, and the database path becomes a URI:

```
database:
  path: "s3://my-bucket/chat.lance"      # or gs://, az://, abfs://

storage:
  aws_endpoint: "http://localhost:4566"  # LocalStack / MinIO / moto
  region: "us-east-1"
  allow_http: "true"
  aws_access_key_id: "test"
  aws_secret_access_key: "test"
```

Standard AWS environment variables (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION`, `AWS_ENDPOINT`, `AWS_ALLOW_HTTP`) are also honoured, so no secrets need be written to disk in production.

11.5 The Smoke Test

The branch adds `tests/object_storage.rs`, a single `#[tokio::test] #[ignore]` case — ignored by default because normal CI has no object store. It connects to an S3-compatible endpoint, runs `init_db`, verifies all five tables are created in the bucket, and writes then reads a row to prove a full round-trip. It reads its target from environment variables:

Variable	Example
`SEAL_TEST_S3_URI`	`s3://seal/it.lance`
`SEAL_TEST_S3_ENDPOINT`	`http://localhost:5001`
`SEAL_TEST_S3_REGION`	`us-east-1`
`AWS_ACCESS_KEY_ID`	`test`
`AWS_SECRET_ACCESS_KEY`	`test`

Running it against `moto` (Docker-free)

```
# A pure-Python S3 mock -- no Docker required.
uv venv /tmp/motoenv
uv pip install --python /tmp/motoenv/bin/python "moto[server]"
/tmp/motoenv/bin/moto_server -p 5001 &

AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1 \
  aws --endpoint-url http://localhost:5001 s3 mb s3://seal

SEAL_TEST_S3_URI=s3://seal/it.lance \
SEAL_TEST_S3_ENDPOINT=http://localhost:5001 \
  cargo test --test object_storage -- --ignored --nocapture
```

The same test works against LocalStack, MinIO, or real S3 by pointing the endpoint and credentials elsewhere.

11.6 A Note on Cargo Features

Talking to cloud stores requires LanceDB's object-store backends to be compiled in. When merging this feature, confirm the `lancedb` dependency in `Cargo.toml` enables the needed backends (e.g. `aws`, `gcs`, `azure`); on `main` the dependency is declared without those features because the connector never reaches for them.